

alternates between even and odd numbers, and the consecutive numbers 4, 5, and 6 appear in the main diagonal.

The legend of the sudden appearance of this simple but fascinating square as a gift from the gods is fitting for the study of algorithms. Algorithms are often easy to verify and use, but they can be difficult to design from scratch. Especially elegant algorithms, when we have the good luck to invent one, seem revelatory, as if they have come out of nowhere as a gift from the gods inscribed on the back of a magical turtle. If you doubt this, try to create an 11×11 magic square from scratch, or try to discover a general-purpose algorithm for generating new magic squares.

Knowledge of this and other magic squares apparently passed from China to Japan at least as early as 1673, when a mathematician named Sanenobu published a 20×20 magic square in Japan. We can create the Luo Shu square in Python with the following command:

```
luoshu = [[4,9,2],[3,5,7],[8,1,6]]
```

It will come in handy to have a function that verifies whether a given matrix is a magic square. The following function does this by verifying the sums across all rows, columns, and diagonals and then checking whether they are all the same:

```
def verifysquare(square):
    sums = []
    rowsums = [sum(square[i]) for i in range(0,len(square))]
    sums.append(rowsums)
    colsums = [sum([row[i] for row in square]) for i in range(0,len(square))]
    sums.append(colsums)
    maindiag = sum([square[i][i] for i in range(0,len(square))])
    sums.append([maindiag])
    antidiag = sum([square[i][len(square) - 1 - i] for i in
\range(0,len(square))])
    sums.append([antidiag])
    flattened = [j for i in sums for j in i]
    return(len(list(set(flattened))) == 1)
```
